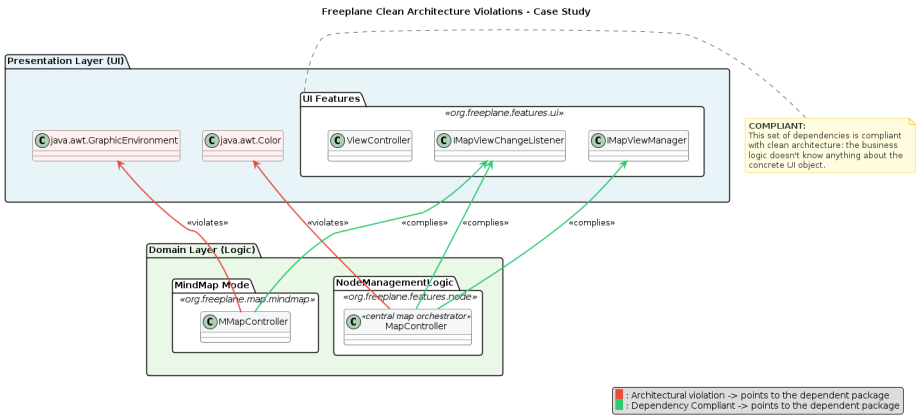
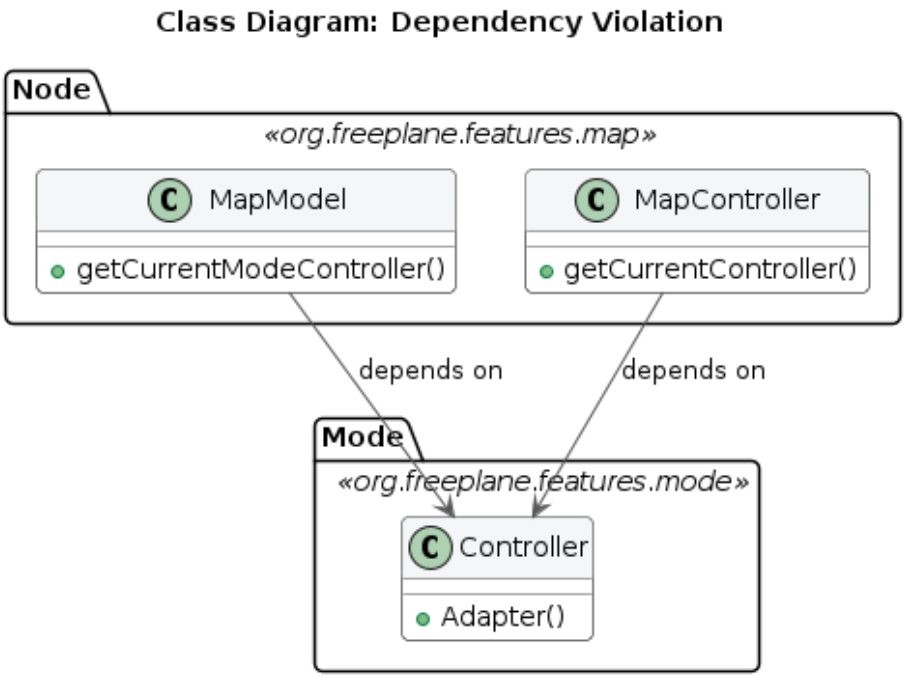
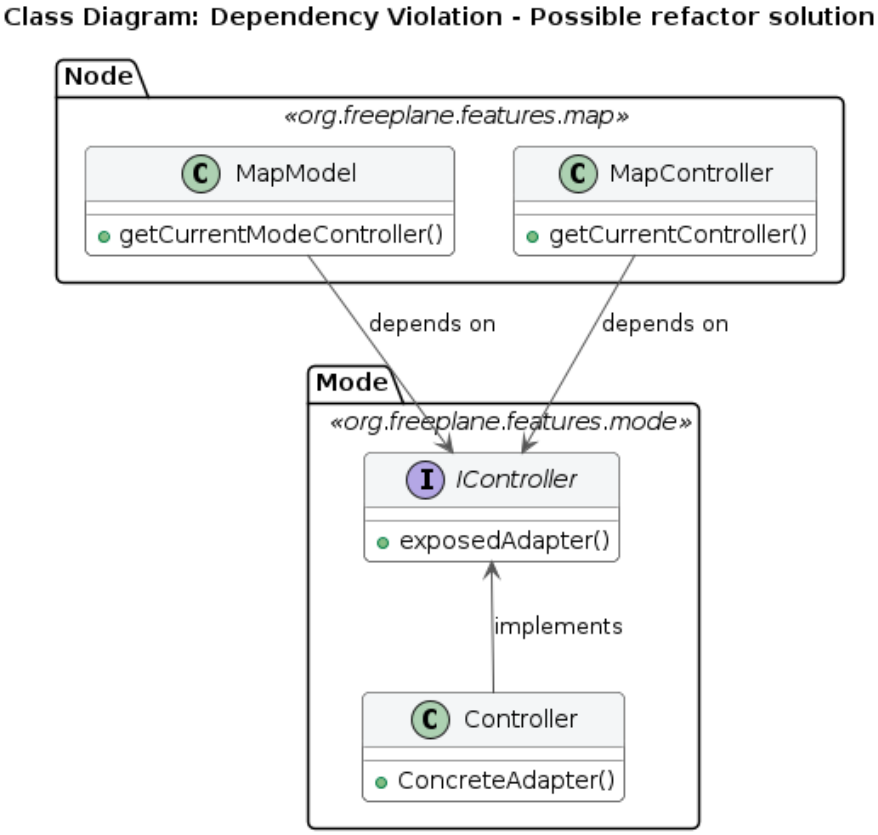
This section aims to clarify the architectural choices made to build the software. The official documentation describes Freeplane as an Extension-object-driven application: objects are built to be extensible from the outside while remaining independent from their extensions.
The Extension-Object pattern is a Design Pattern defined by Erich Gamma in 1998. However, it provides a useful starting point to better understand how it fits in the broader architectural environment.
The first step was to define *entities*, *use cases* and *external layers*. To figure out a starting point, Freeplane developers were contacted, and they provided an AI tool, deepwiki.com, to ask questions to gather information about the codebase. The chatbot gave us a hint: entities could be found in many packages. `org.freeplane.features.map` was among them. This package is responsible for the definition of basic business rules for managing nodes and maps.
To double check, a statistical analysis was carried out: data from commits throughout the software's lifecycle was extracted, and metrics were calculated: a stability measure was computed, considering average time between commits, number of commits and package age. This first analysis returned unexpected results: `org.freeplane.features.map` is a very unstable component, with a commit every 3 days on average.
This can be explained by the high coupling between the package and UI components. A deeper analysis reveals that almost 43% of commits share changes with freeplane UI components, and this number grows if we look at subpackages such as `org.freeplane.features.map.filemode` or `org.freeplane.features.map.clipboard` (both at almost 61% of shared commit number with ui components).
Code analysis reveals that most classes in the package have dependencies on visual interface, both on custom Freeplane UI classes and standard Java AWT ones. The approach is different: while dependencies towards `org.freeplane.ui` packages respect Clean Architecture rules, such as dependencies ruled by Interfaces, java UI standard classes are directly imported, thus violating the same principles developers tried to respect with custom packages.



There are other crucial violations of the Clean Architecture pattern: there is no clear definition of *entities*, *use cases* and other layers. Some classes may look similar to one of these concepts: `MapModel` could be classified as an *entity*, `MapController` as a *use case*, `Controller` from `org.freeplane.features.mode` as an *adapter*. Most critical violations of the Clean Architecture pattern can be found in the dependency among these three classes: `MapModel` and `MapController` depend on the `Controller`; the flow of dependencies is broken and therefore inner business logic cannot be tested in isolation.






Compliance with the principles from the Clean Architecture pattern can be found in the *persistence layer*: classes such as `MapReader` and `MapWriter` are at the outer layer of the architecture, and there are no dependency violations. These architectural flaws can be extended to the application as a whole: the analysis of the `org.freeplane.features.map` package is a case study which can fairly represent the overall structure of the software. Freeplane does not comply with modern architectural patterns and particularly with the Clean Architecture one, and it can be explained by the age of the software, whose development started in 2009.